

Lab 6: Final-Project Integration

Subsystem Assembly, Programmed Behavior, and Testing

ENGR 1105 | Fall 2026

Name	Team
Partner(s)	Date

Learning Objectives

By the end of Lab 6, your team should be able to:

1. convert a project idea into measurable success criteria;
2. create a system block diagram, I/O table, pin budget, and power plan;
3. test the mechanical, power, input, controller, and actuator subsystems separately;
4. organize Arduino code into readable functions and operating states;
5. integrate one subsystem at a time using safe startup behavior;
6. use thresholds, hysteresis, timeouts, or repeated confirmation when appropriate;
7. create and execute a test matrix; and
8. document failures, troubleshooting actions, and design changes.

1. Lab Organization

Lab 6 is one graded lab completed across two laboratory meetings.

Meeting	Main work
Lab 6.1	Define project behavior, plan interfaces, assign pins and power, assemble subsystems, and demonstrate each subsystem separately
Lab 6.2	Organize program behavior, integrate subsystems incrementally, execute a test matrix, troubleshoot, and document the integrated prototype

Project Safety

1. Disconnect USB and external power before major wiring changes.
2. Begin every program with actuators stopped or at a safe position.
3. Verify voltage, polarity, current capability, and common ground.

4. Keep fingers, hair, clothing, cables, and loose objects away from moving parts.
5. Do not intentionally stall a motor or servo.
6. Stop immediately if a component becomes hot, smells unusual, smokes, resets repeatedly, or moves unexpectedly.
7. Test at low speed and with limited motion before using the full mechanism.

2. Required Deliverables

Check	Required item
☐	Project purpose and measurable success criteria
☐	System block diagram and interface table
☐	Pin budget, power plan, and wiring plan
☐	Separate subsystem test records
☐	State table or behavior sequence
☐	Integrated Arduino program
☐	Test matrix with actual results
☐	Troubleshooting and design-change record
☐	Final reflection and readiness checklist

Lab 6.1

Project Planning and Subsystem Assembly

Thursday, December 3, 2026

3. Part A: Define the Project

Project Identity

Item	Team response
Project name	
Robot or mechatronic-system type	
Primary physical behavior	
Sensor or user input	
Actuator or output	
Mechanical structure	
Power source(s)	

One-Sentence Purpose

Complete this sentence:

Our project will use _____ to detect or receive _____, decide _____, and produce _____.

Measurable Success Criteria

Write criteria that can be observed or measured.

Success Criteria

No.	The project is successful when...	Measurement or evidence
1		
2		
3		
4		

Checkpoint 1: Purpose and Success

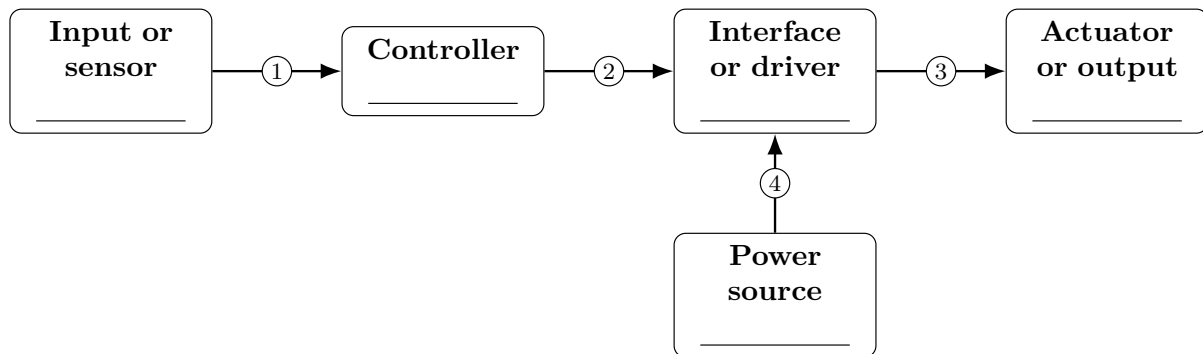
Explain why “the robot works” is not a sufficiently precise success criterion.

4. Part B: Identify Constraints

Constraint category	Project-specific constraint
Available time	
Available components	
Supply voltage and current	
Size or workspace	
Mechanical motion limits	
Safety	
Team knowledge or skills	

5. Part C: Create the System Block Diagram

Use the project-specific labels in the blank diagram.



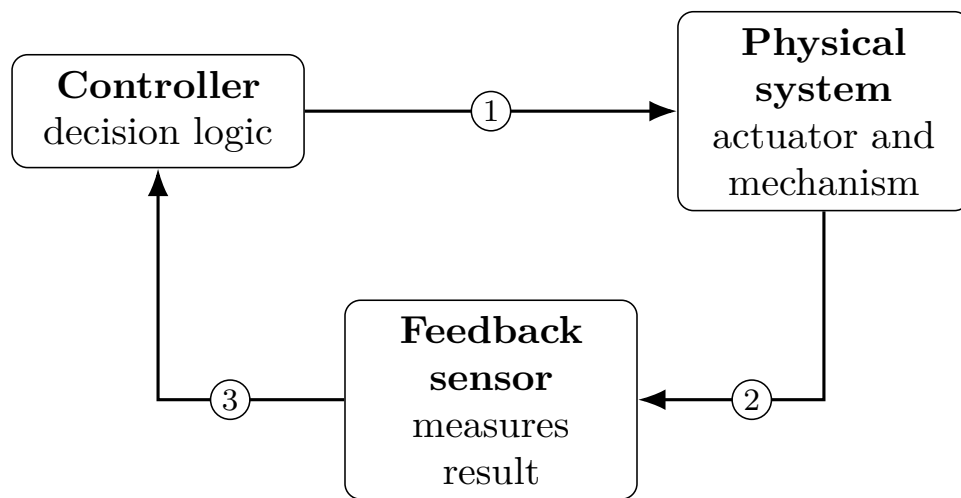
1 measurement or user input

2 logic command

3 actuator power or output signal

4 power input

Add feedback when the controller uses information about the physical result:



1 command

2 measured physical result

3 feedback information

Does your project use feedback about the physical result?

☐ yes

☐ no

☐ partially or indirectly

Explain:

6. Part D: Create the Interface Table

An interface is a connection between two subsystems.

Interface Table

No.	From	To	Signal or power	Important requirement
1				
2				
3				
4				
5				
6				

Possible requirements include:

- correct voltage;
- common ground;
- PWM-capable pin;
- input-only or output-only direction;
- pull-up resistor;
- external motor supply;
- mechanical alignment;
- safe motion range.

7. Part E: Create the I/O and Pin Budget

Pin Budget

Device or signal	Arduino pin	Input/output	Signal type	Notes
------------------	-------------	--------------	-------------	-------

Checkpoint 2: Interface Risk

Identify the project interface most likely to cause a failure. Explain why.

8. Part F: Create the Power Plan

Power Budget

Device	Voltage	Typical current	Starting or peak current	Supply source
Arduino				
Sensor/input				
Actuator 1				
Actuator 2, if used				
Driver/interface				
Other				

Answer:

1. Which component is expected to draw the largest current?

2. Which grounds must be connected to create a shared reference?

3. What should happen if the actuator supply is disconnected?

Power Review

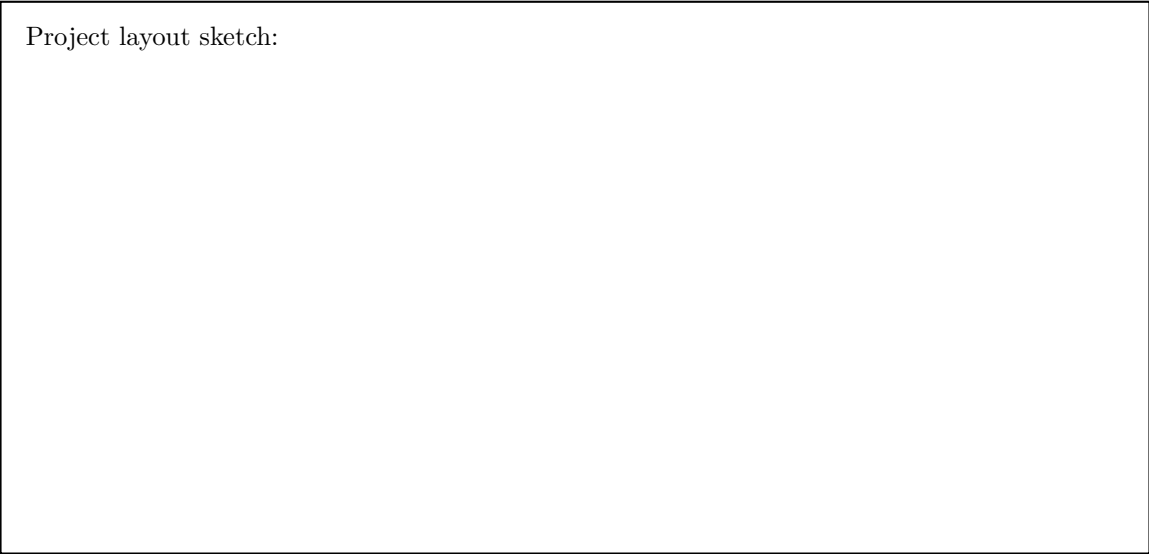
Do not apply actuator power until the team can identify:

- the positive and ground terminals;
- the rated voltage;
- the driver or interface;
- the common-ground connection;
- the expected safe startup state.

9. Part G: Mechanical Layout and Wiring Plan

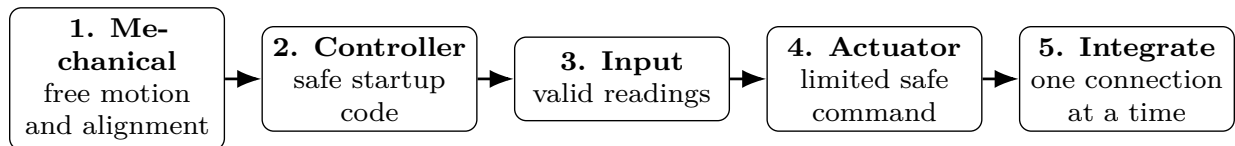
Sketch the top or side view of the project. Include the Arduino, sensor/input, actuator, driver, power source, moving parts, and major cable paths.

Project layout sketch:



10. Part H: Subsystem Bring-Up Sequence

Do not connect everything at once.



11. Part I: Mechanical Subsystem Test

Test with electrical power disconnected.

Mechanical Test Record

Check	Pass?	Observation or correction
Structure remains stable		
Moving parts travel freely		
No wire enters moving region		
Actuator is securely mounted		
Sensor is correctly oriented		
Motion stops before collision		

12. Part J: Controller and Power Test

Upload a safe-startup program before actuator power is connected.

```

1 void setup() {
2   Serial.begin(9600);
3
4   // Configure all input and output pins here.
5
6   // Set every actuator command to STOP
7   // or to a known safe position.
8 }
9
10 void loop() {
11   // No automatic motion during this test.
12 }
  
```

Controller and Power Test

Observation	Pass?	Notes
Program uploads successfully		
Arduino starts without repeated reset		
Actuator remains stopped or safe		
Logic supply voltage is correct		
Actuator supply polarity is correct		
Ground reference is connected		

13. Part K: Input or Sensor Subsystem Test

Test the input without automatic actuator motion.

```

1 void loop() {
2   // Replace this with the project input.
3   int inputValue = analogRead(A0);
4
5   Serial.print("input=");
6   Serial.println(inputValue);
7
8   delay(50);
9 }

```

Input Test

Test condition	Expected reading	Actual reading	Valid and repeatable?
Minimum condition			
Middle condition			
Maximum condition			
Boundary condition			
Invalid or disconnected input			

Describe noise, unstable values, invalid data, or sensitivity to placement:

14. Part L: Actuator Subsystem Test

Command only a limited, safe behavior.

Actuator Test

Command	Expected motion	Actual motion	Notes
Stop or safe position			
Low command			
Middle command			
Limited direction or angle change			
Return to stop or safe position			

Checkpoint 3: Separate Subsystems

At the end of Lab 6.1, list each subsystem that operates correctly by itself and each subsystem that still needs work.

15. Lab 6.1 Shutdown

Check	Required action
☐	Command actuator stop or safe position.
☐	Disconnect actuator power.
☐	Disconnect USB.
☐	Photograph or sketch all important wiring.
☐	Save the current program with a clear version name.
☐	Record unresolved problems for Lab 6.2.

Lab 6.2

Programmed Behavior, Integration, and Testing

Friday, December 4, 2026

16. Part M: Define Operating States

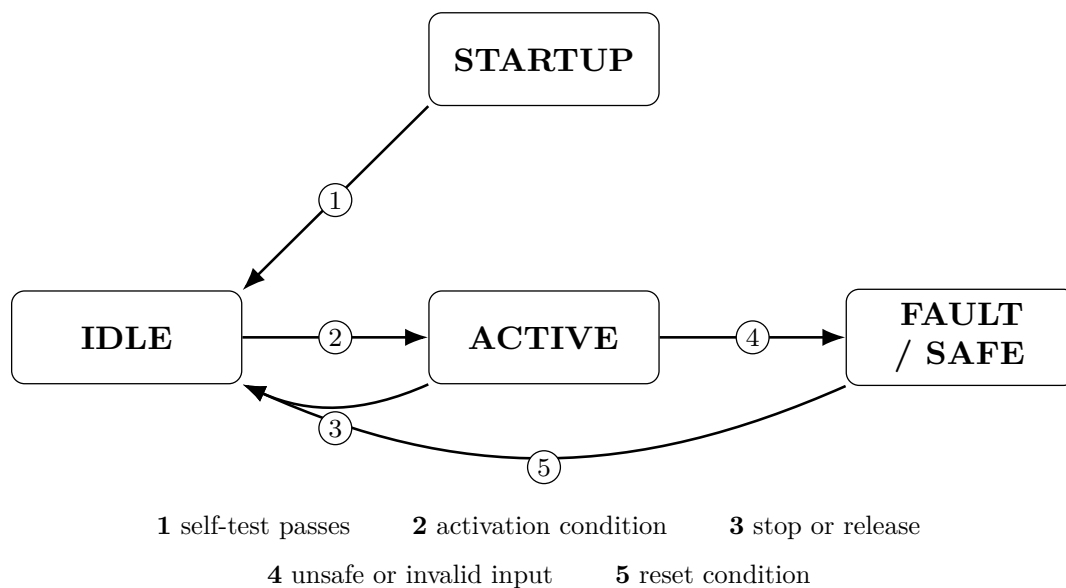
Most projects are easier to understand when behavior is divided into named states.

State Table

State name	Entry condition	Actuator/output behavior	Exit condition
STARTUP	power applied or reset		
IDLE			
ACTIVE			
WARNING, if used			
FAULT or SAFE	invalid or unsafe condition		
Other:			

17. Part N: Draw the State Diagram

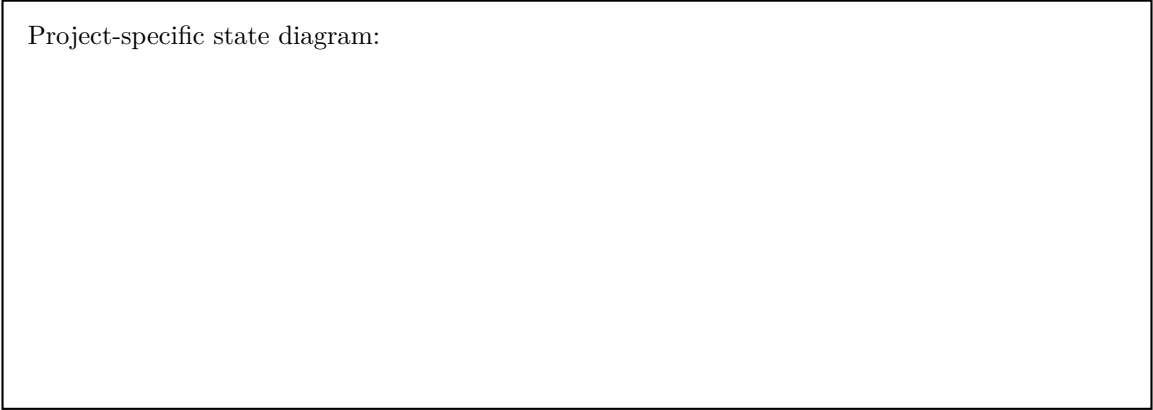
Use arrows to show permitted transitions and label the transition conditions.



Any operating state may enter FAULT / SAFE when an unsafe or invalid condition is detected.

Modify the diagram or redraw it below for your project:

Project-specific state diagram:



18. Part O: Organize the Arduino Program

Create functions that separate sensing, decisions, actuator commands, and diagnostics.

```
1  enum State {
2      STARTUP,
3      IDLE,
4      ACTIVE,
5      FAULT
6  };
7
8  State state = STARTUP;
9
10 void readInputs() {
11     // Read sensor or user input.
12     // Check validity and update variables.
13 }
14
15 void updateState() {
16     // Decide which state should be active.
17 }
18
19 void applyOutputs() {
20     // Command motors, servos, LEDs, or sound.
21     // FAULT must command a safe output.
22 }
23
24 void printDiagnostics() {
25     // Print input, state, and output command.
26 }
27
28 void setup() {
29     Serial.begin(9600);
30
31     // Configure pins.
32     // Begin stopped or at a safe position.
33 }
34
35 void loop() {
36     readInputs();
37     updateState();
38     applyOutputs();
39     printDiagnostics();
40 }
```

Program Function Plan

Function name	Purpose in your program
<code>readInputs()</code>	
<code>updateState()</code>	
<code>applyOutputs()</code>	
<code>stopAll()</code> or safe-output function	
<code>printDiagnostics()</code>	
Other function:	

19. Part P: Define Safe Defaults

Complete the statements:

1. On power-up, the project will:

2. If the sensor or input is invalid, the project will:

3. If a timeout occurs, the project will:

4. If the actuator or mechanism behaves unexpectedly, the operator will:

20. Part Q: Improve Decision Reliability

Select the methods that apply to the project.

Use?	Method	Project application
☐	Hysteresis	Use separate activation and deactivation thresholds.
☐	Debouncing	Ignore rapid changes from a mechanical button or switch.
☐	Timeout	Treat a missing or delayed sensor response as invalid.
☐	Repeated confirmation	Require multiple consistent readings before changing state.
☐	Moving average	Reduce short random variation in sensor values.
☐	Rate limiting	Prevent abrupt actuator-command changes.

Record the selected values:

Reliability Settings

Parameter	Selected value or rule
Activation threshold	
Deactivation threshold	
Timeout	
Required confirmations	
Filter window, if used	
Maximum output command	
Rate limit, if used	

Checkpoint 4: Reliable Decision

Explain one situation in which a single raw sensor reading could produce an incorrect project response. State the protection method used.

21. Part R: Avoid Long Blocking Delays

Use `millis()` when the project must read inputs while timing an action.

```
1 unsigned long previousTime = 0;
2 const unsigned long interval = 250;
3
4 void loop() {
5     unsigned long now = millis();
6
7     readInputs();
8     updateState();
9
10    if (now - previousTime >= interval) {
11        previousTime = now;
12
13        // Perform a timed update without stopping
14        // the rest of the program.
15    }
16
17    applyOutputs();
18 }
```

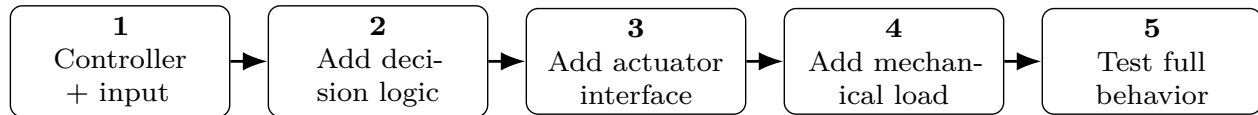
Does your project require responsive timing while another action continues?

☐ yes ☐ no

Explain:

22. Part S: Incremental Integration

Connect and verify one interface at a time.



Integration Log

Step	Newly connected item	Worked?	Observation or correction
1	Controller + input		
2	Decision logic		
3	Actuator interface		
4	Mechanical load		
5	Full behavior		

23. Part T: Serial Diagnostic Output

Print only the values needed to understand behavior.

```

1 Serial.print("input=");
2 Serial.print(inputValue);
3
4 Serial.print(", valid=");
5 Serial.print(inputValid);
6
7 Serial.print(", state=");
8 Serial.print(state);
9
10 Serial.print(", command=");
11 Serial.println(outputCommand);
  
```

Record one diagnostic line for each important state:

Diagnostic Evidence

State or condition	Example Serial output
STARTUP	
IDLE	
ACTIVE	
WARNING, if used	
FAULT or invalid input	

24. Part U: Create the Test Matrix

A useful test has an initial condition, input, expected result, and actual result.

No.	Initial condition	Input or event	Expected result	Actual result	Pass?
1	power off	apply power			
2	IDLE	minimum input			
3	IDLE	typical input			
4	IDLE	activation boundary			
5	ACTIVE	deactivation boundary			
6	ACTIVE	rapid input variation			
7	any state	invalid or disconnected input			
8	ACTIVE	mechanical load or resistance			
9	any state	emergency stop or safe command			
10	repeated trial	repeat full sequence five times			

25. Part V: Boundary Tests

Record values just below, at, and just above each important threshold.

Boundary Test

Condition	Input value	Expected state	Actual state or behavior
Below activation threshold			
At activation threshold			
Above activation threshold			
Below deactivation threshold			
At deactivation threshold			
Above deactivation threshold			

26. Part W: Troubleshooting Log

When a failure occurs, change only one item at a time.

Troubleshooting Record

No.	Observed problem	Suspected layer	Change or test	Result
1				
2				
3				
4				
5				

Layers may include:

- mechanical;
- power;
- wiring or interface;
- sensor or input;
- actuator or driver;
- code or state logic;
- timing or communication.

Checkpoint 5: Fault Isolation

Choose one failure from the log. Explain how the team determined the layer that caused it and why the final correction worked.

27. Part X: Repeated Reliability Test

Run the complete intended behavior at least five times.

Repeated Trials

Trial	Completed behavior?	Failure or variation observed	Notes
1			
2			
3			
4			
5			

Reliability result:

$$\frac{\text{successful trials}}{\text{total trials}} = \text{———} = \text{———} \%$$

28. Part Y: Design Changes

Record changes made during integration.

Design-Change Record

No.	Original design	Change made	Evidence that the change helped
1			
2			
3			
4			

29. Part Z: Final Integration Readiness

Check	Readiness requirement
☐	Structure is stable and moving parts have clearance.
☐	Sensor or user input produces valid and repeatable values.
☐	Actuator operates safely through the required range.
☐	Voltage, polarity, driver, current capability, and common ground are correct.
☐	Program begins in STARTUP, IDLE, or another safe state.
☐	Invalid input leads to a safe output.
☐	State changes occur at the intended thresholds or events.
☐	The project stops safely when commanded.
☐	Full behavior succeeds repeatedly.
☐	Wiring is secure and clear of moving parts.
☐	Code, diagram, test results, and design changes are saved.

Before Full-Speed or Full-Range Operation

A team member must be ready to disconnect actuator power. Test the mechanism unloaded or lightly loaded before attempting the final operating condition.

30. Final Demonstration

Demonstrate:

Check	Demonstration requirement
☐	Project starts safely.
☐	Input or sensor is shown and explained.
☐	Controller decision or state is shown and explained.
☐	Actuator creates the intended physical behavior.
☐	Project responds correctly at a boundary condition.
☐	Invalid or unsafe input produces a safe result.
☐	Team shows diagnostic evidence or test results.
☐	Each student can explain one mechanical, electrical, and software feature.

31. Final Reflection

Answer in complete sentences.

1. Describe the complete sense-decide-act sequence in your project.

2. Which subsystem was easiest to test separately? Why?

3. Which interface caused the greatest integration difficulty? Explain.

4. How does the project behave when the input is invalid or unsafe?

5. Identify one example of feedback, threshold logic, state-based logic, or timed behavior in the project.

6. What test provided the strongest evidence that the project is reliable?

7. State one improvement that should be completed before the final demonstration.

32. Submission Checklist

Check	Required item
☐	Project purpose and success criteria are complete.
☐	Block diagram, interface table, pin budget, and power plan are complete.
☐	Mechanical, controller, input, and actuator tests are documented.
☐	State table or behavior sequence is complete.
☐	Arduino code uses clear functions and safe defaults.
☐	Integration log and diagnostic evidence are complete.
☐	Test matrix, boundary tests, and repeated trials are complete.
☐	Troubleshooting and design changes are documented.
☐	Final reflection is complete.
☐	Current code and project files are submitted as instructed.

Power Down Before Leaving

Command all actuators to stop or a safe position. Disconnect actuator power, disconnect USB, secure the prototype, and store the current project files.